AI Engineering Harness

Architecture Design Document

Drug Interaction Checker — Track B (React / Frontend)

Intrahealth / HEALWELL AI — Interview Task 002

Track	Track B — Frontend (React). The harness delivers a clinician-facing Drug Interaction Checker as a React component consuming FHIR data from Medplum's API.
Core Design Principle	The harness transforms the traditional Software Development Life Cycle into an autonomous, stateful pipeline. Rather than a single monolithic coding AI, specialised agents mirror the steps a software engineer naturally works through when tackling a ticket. Orchestration logic lives in the graph, not in an LLM.
Approach	Design-heavy with a partial prototype. Two agents are fully implemented (Issue Analyzer, Medplum Expert). The remaining pipeline is designed in detail with clear extension points.

Contents

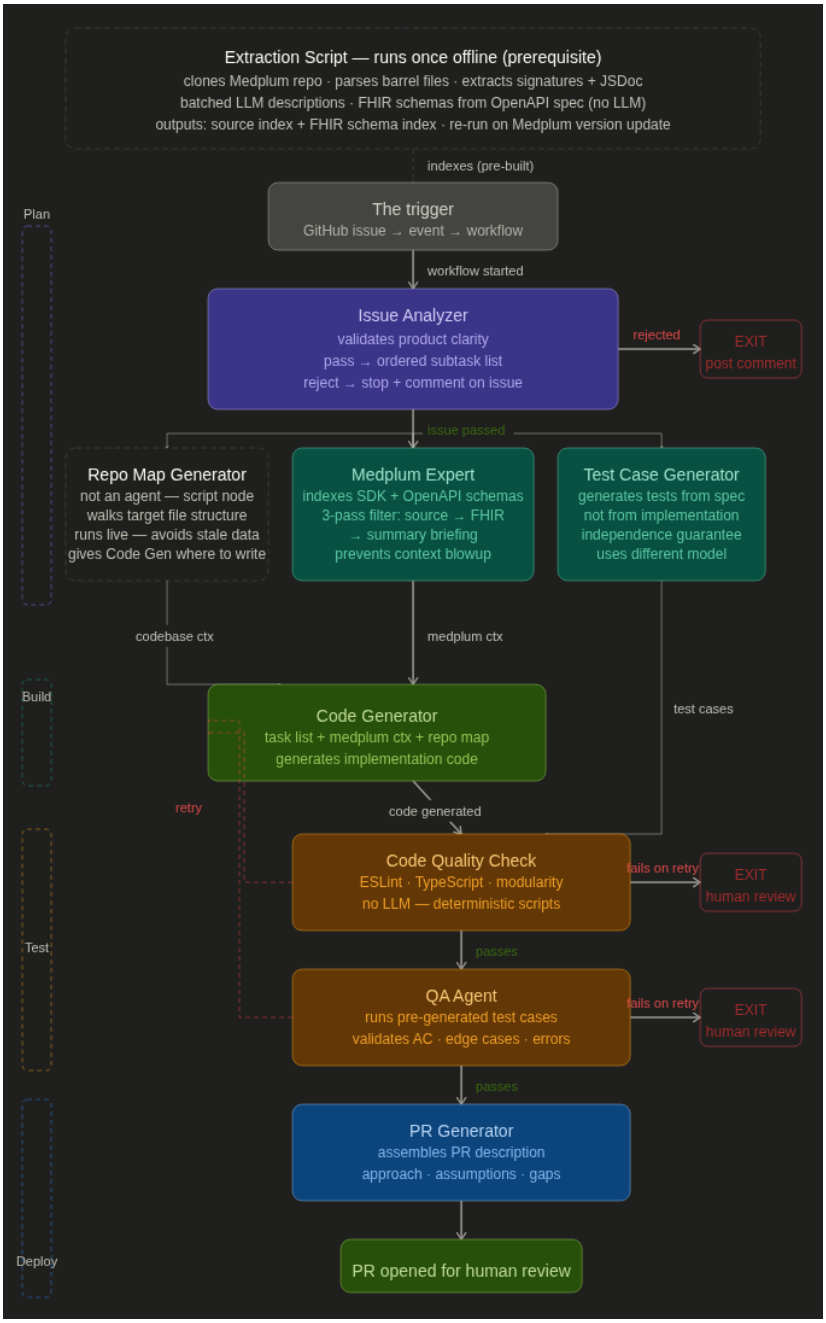
1. Overview 3
   1.1 System Diagram 3
   1.2 Pipeline Overview 4
   1.3 State Object 5
   1.4 Submission Scope 5
2. Technology Stack 6
3. Agent Specifications 7
   3.1 Issue Analyzer 7
   3.1.1 The Anatomy of an Actionable Ticket 8
   3.2 Medplum Expert 9
   3.3 Test Case Generator 11
   3.4 Code Generator 12
   3.5 Code Quality Check 13
   3.6 QA Agent 13
   3.7 PR Generator 15
4. Testing 17
   4.1 Unit Tests 17
   4.2 Graph Wiring Tests 17
   4.3 Groundedness and Structural Integrity Suites 18
   4.4 Integration Runners 18
   4.5 Test Philosophy 18
5. Medplum Context Design Rationale 20
   5.1 Why Not RAG 20
   5.2 Why Not Custom MCP 20
6. Quality Gates 20
   6.1 Automated Checks 20
   6.2 Self-Correction 20
   6.3 Test Independence 20
   6.4 Human Handoff 21
7. Failure Modes and Observability 21
   7.1 Observability 21
8. Deliberate Tradeoffs 22
9. Future Work 22

1. Overview

This document describes the architecture of an AI engineering harness. The system takes a GitHub Issue describing a frontend feature and autonomously delivers a tested pull request without a human writing implementation code. While a drug interaction checker serves as the primary work order to prove the system, the harness itself is the core innovation.

The harness is built around a stateful multi-agent workflow powered by LangGraph. Specialized agents progressively build a shared HarnessState object. Each node in the graph reads specific required data, writes a typed partial update, and then passes control to the next node through deterministic routing logic.

1.1 System Diagram



1.2 Pipeline Overview

The pipeline is a LangGraph state graph with conditional routing, parallel fan-out, and bounded retry loops. The graph structure encodes all orchestration logic because no LLM decides what runs next. A single HarnessState object flows through all nodes. Each agent reads from this object and writes a typed partial update back to it. While the current implementation runs sequentially from the Issue Analyzer to the Medplum Expert, the parallel branch and retry loop are already designed in the full graph spec.

Step	Agent	Status	Responsibility	Key Design Decision
prereq	Extraction Script	✅ Built	Walks Medplum repo, generates source index + FHIR schema index	Not an agent; Runs once offline; re-run on Medplum version update
1	Issue Analyzer	✅ Built	Validates ticket, produces ordered subtask list or rejects with clarification comment	Reject early, save compute. False rejection beats false pass.
2a	Repo Map Generator	🛠 Planned	Script node; walks target repo file structure live to tell Code Generator where to write	Not an agent; runs live to avoid stale data
2b	Medplum Expert	✅ Built	Three-pass filter: source entries, FHIR schemas, context summary	Static indexes + targeted injection = no context blowup
2c	Test Case Generator	🛠 Planned	Generates tests from spec before code is written; uses a different model than the Code Generator (e.g. Gemini vs Claude) to avoid shared blind spots	Independence guarantee; tests cannot be written to match code that does not exist yet
3	Code Generator	🛠 Planned	Implements React components from subtask list, Medplum context, and repo map; retry-eligible	Core value step
4	Code Quality Gate	🛠 Planned	ESLint, tsc, modularity — deterministic, no LLM; on retry failure exits to human	Machine-readable errors feed retry loop to Code Generator
5	QA Agent	🛠 Planned	Runs pre-generated test cases; validates AC, edge cases, errors; on retry failure exits to human	Tests derived from spec, not implementation

Step	Agent	Status	Responsibility	Key Design Decision
6	PR Generator	🛠 Planned	Assembles PR description: approach, assumptions, Known Gaps section	Human handoff

1.3 State Object

State grows incrementally as agents complete. Fields marked optional are populated only by the agent responsible for them: no agent receives context it cannot use. The planned fields below reflect the full pipeline design; only the implemented fields are currently populated at runtime.

```
interface HarnessState {
  issue: GitHubIssue;                // raw GitHub issue payload
  subtasks: Subtask[];               // ordered task list from Issue Analyzer
  repoMap: RepoMapEntry[];                // target codebase structure from Repo Map Generator
  status: PipelineStatus;            // running | needs_clarification | done | failed
  rejectionReason?: string;          // populated only on reject
  medplumContext?: MedplumContextDocument;  // assembled by Medplum Expert
  testCases?: TestCase[];            // generated from spec before code is written
  generatedCode?: GeneratedFile[];       // files from Code Generator
  qualityCheckResult?: QualityCheckResult;  // ESLint, tsc, modularity results
  qaResult?: QAResult;               // test run results, AC validation
  logs: LogEntry[];                  // observability trail
}
```

1.4 Submission Scope

The intended MVP covered four agents: Issue Analyzer, Medplum Expert, Code Generator, and PR Generator. These were scoped down from the full seven-agent pipeline. The Issue Analyzer and Medplum Expert were built first because they establish the foundational capabilities everything else depends on. Building front-to-back meant each could be tested against real inputs before the next was designed.

Agent	Status	Notes
Issue Analyzer	✅ Built	Fully implemented with Zod structured output, product-vs-technical-gap classification, and a 7-case test suite
Medplum Expert	✅ Built	Three-pass LLM filter with source index + FHIR schema index, source snippet extraction, and context summary generation
Code Generator	🛠 Planned	Design documented — receives subtasks, Medplum context, and repo map; outputs typed file JSON to target project
PR Generator	🛠 Planned	Assembles PR description from all run artifacts; includes Known Gaps section for human reviewer

2. Technology Stack

Component	Choice	Rationale
Orchestration	LangGraph TypeScript	Stateful graph execution; routing logic lives in graph structure, not in an LLM
LLM Abstraction	LangChain Core	Dev on Gemini, ship on Anthropic — provider swap is a one-line config change, no agent code touched.
Schema Validation	Zod	Structured LLM output validation; typed, predictable agent responses
LLM (production)	Claude Haiku / Sonnet	Haiku-equivalent for lighter agents (Issue Analyzer, Medplum Expert, PR Generator); Sonnet-equivalent for Code Generator where capability matters most
LLM (dev/iteration)	Gemini Flash / Pro	Swappable via getModel() env var; Flash for lighter agents, Pro for Code Generator — mirrors the production model tier split
Quality Gates	ESLint, tsc, Vitest	Deterministic, zero-cost, machine-readable error output
Trigger (planned)	GitHub Actions	Prototype runs locally with issue.json; GH Actions wires the full loop

3. Agent Specifications

3.1 Issue Analyzer

Model	Claude Haiku (prod) / Gemini Flash (dev)
Status	✅ Built — Gate 1, nothing proceeds without passing

The Issue Analyzer is a strict requirements reviewer. Its job is to decide whether a ticket is clear enough for autonomous implementation. It does not write code or suggest implementation approaches.

On PASS: produces an ordered subtask list with titles, descriptions, acceptance criteria, and explicit dependencies.

On REJECT: produces a plain-language GitHub comment explaining exactly what is missing. This is written to the ticket author with no mention of internal tooling.

The classification step

An early challenge involved the agent rejecting valid tickets by misidentifying technical gaps, such as which FHIR resource type to use, as product gaps. Technical resolution is the specific responsibility of the Medplum Expert downstream. Rejecting tickets for these reasons wastes perfectly actionable work orders.

The solution was a classification step in the system prompt that forces the agent to distinguish between product gaps and technical gaps before making a decision. This step includes concrete clinical and FHIR examples for each category. Both question types now behave correctly: product gaps trigger a rejection while technical gaps pass through to the next node.

Structured output schema

```
AnalyzerResponseSchema = z.object({
  decision: z.enum(["pass", "reject"]).describe("Whether the ticket passed or was rejected due to product gaps."),
  reasoning: z.string().describe("Internal note on why this passes or fails the product clarity bar."),
  subtasks: z.array(
        z.object({
              title: z.string().describe("Short task title"),
              description: z.string().describe("What this task achieves and why."),
              acceptanceCriteria: z.array(z.string()).describe("List of testable behaviours"),
              dependencies: z.array(z.string()).describe("List of subtask titles this task depends on, or empty array if none"),
        })
  ).optional().describe("Provide subtasks ONLY if the decision is 'pass'."),
  comment: z.string().optional().describe("The GitHub comment to post to the reporter. Provide ONLY if the decision is 'reject'."),
});
```

3.1.1 The Anatomy of an Actionable Ticket

The harness operates on a "Garbage In, Garbage Out" principle. Since the Issue Analyzer is the terminal gate for the planning phase, the quality of the initial GitHub Issue dictates the success of every downstream agent.

The Standard Ticket Structure

To pass, a ticket must follow this structured schema. Each section provides the "mechanical necessity" required by a specific downstream consumer.

Section	Purpose	Primary Consumer
Summary	Establishes high-level context and the "why."	Medplum Expert (for context)
User Story	Provides persona context for edge-case reasoning.	Issue Analyzer (for scope)
Acceptance Criteria (AC)	A binary checklist of observable behaviors.	Test Case Generator (Crucial)
Domain Context	Hidden rules, business logic, and severity definitions.	QA Agent (Phase 2)
Out of Scope	Hard boundaries to prevent AI over-engineering.	Issue Analyzer
Technical Notes	Platform constraints (e.g., specific hooks or themes).	Code Generator

Heuristics for Success

1. The "Behavior vs. Implementation" Rule

A ticket should describe what the user sees and how the system reacts, never the specific code to achieve it.

- ❌ **Bad:** "Use a useEffect to fetch patient data and map it to a useState array."
- ✅ **Good:** "The system must display the patient's active medication list upon component load."
- **Why:** If the ticket dictates the code, the Code Generator loses its ability to optimize for the existing Medplum patterns found in the source index.

2. The "Deterministic AC" Rule

Acceptance Criteria must be binary (Pass/Fail). If a human can't agree on whether a criterion is met, an LLM certainly can't.

- ❌ **Bad:** "The UI should feel fast and modern."
- ✅ **Good:** "A loading skeleton must be displayed if the API response takes longer than 200ms."

3. Product vs. Technical Clarity

As noted in the Classification Step, the agent will reject tickets with "Product Gaps."

- **To avoid REJECTION**: Ensure business logic (like alert severity levels or dismissal rules) is explicit.
- **Technical Gaps are OK:** You do not need to name specific FHIR resources (like MedicationRequest); the harness is designed to resolve those technical ambiguities downstream.

Integration with State

Once validated, the Issue Analyzer transforms this prose into the subtasks array. These subtasks carry the original Acceptance Criteria forward, ensuring that the Test Case Generator is always testing against the author's intent, rather than a hallucinated interpretation.

3.2 Medplum Expert

Model	Claude Haiku (prod) / Gemini Flash (dev) — three LLM passes per run
Status	✅ Built — answers the codebase context challenge

> The Medplum Expert serves as the bridge between feature requirements and code generation. It reads the subtask list and identifies exactly which components, hooks, and FHIR resource types are needed. This ensures the Code Generator receives targeted and accurate context instead of having to reason about the entire Medplum SDK from scratch.

The codebase context challenge — compression pipeline

The brief asks specifically: how does the harness give agents enough context about Medplum's React patterns, component conventions, and data hooks without flooding the context window? The answer is a compression pipeline that reduces the full Medplum monorepo down to a focused, targeted context document:

Layer	What it produces	Size
Raw Medplum repo	Full monorepo source	~200 MB
Extraction scripts (run once)	Structured source index + FHIR schema index	~3000 KB
Pass 1 + 2 filter (LLM)	Relevant components, hooks, FHIR resource types	~20 KB
Pass 3 assembly (LLM)	MedplumContextDocument — targeted, precise context	~5 KB
Code Generator prompt	What the model actually sees	Focused, no noise

The Code Generator never sees 150 FHIR schemas. It sees 3. It never sees 80+ components. It sees 5–10. This is how the context challenge is solved without RAG infrastructure.

Knowledge base — how it is built

Two extraction scripts run once against the cloned Medplum repo:

- build:index:source — walks packages/react, packages/react-hooks, and packages/core. Barrel-aware: parses index.ts files first, resolves re-exports, then extracts signatures from actual source files. Uses batched LLM calls in groups of 8 with withStructuredOutput to generate developer-focused descriptions and 3 to 6 keyword tags per entry, such as resource types, UI patterns, and use cases. These tags are what make the index scannable for the Pass 1 filter. Output: medplum-source-index.json
- build:index:fhir — reads the official OpenAPI spec at api.medplum.com/openapi.json, which contains all 722 FHIR resource schemas. Most accurate and deterministic source available. Output: medplum-fhir-schemas.json

A key implementation challenge: initial signature extraction was missing ~90% of the public API because Medplum hides everything behind barrel exports. The scraper was rewritten to be barrel-aware, resolving the internal re-export chain before extracting signatures.

Three-pass LLM filter

Pass	Input	Output	Schema
1 — Source filter	Subtask list + full source index summary (one line per entry)	Which components, hooks, utilities are relevant	{ selectedIds: string[] }
2 — FHIR filter	Subtask list + full FHIR schema names	Which FHIR resource types are needed	{ selectedResourceNames: string[] }
3 — Context summary	Selected source snippets + selected FHIR schemas	Narrative briefing for Code Generator	MedplumContextDocument

Pass 1 and 2 use LLM judgement as a cheap filter over a large flat list. Pass 3 synthesises the filtered material into a targeted briefing. The Code Generator receives focused context rather than noise.

Source snippets: bridging selection and generation

The index gives the filter enough to select the right components, but a description alone does not tell the Code Generator how to use them. Knowing that HumanNameInput exists and what it does is not the same as knowing its prop interface. Without the actual API surface, the Code Generator is guessing.

To bridge this, the Medplum Expert reads up to 80 lines of actual source from each selected entry and attaches them to the MedplumContextDocument. The snippets carry the real TypeScript: prop interfaces that show how components are composed, hook signatures that show data access patterns, and JSDoc that captures intended usage. This is how Medplum's component conventions and UI primitives are

conveyed. They are provided not as a written guide, but as the actual API surface needed to use them correctly. Only selected entries are read, which keeps the context focused.

Current limitation: During implementation, snippets were read from a locally cloned Medplum repo at runtime via MEDPLUM_REPO_PATH rather than stored in the index at build time. This was an oversight because the repo is already present during the extraction step. The fix is straightforward: store the snippet as a field on each SourceIndexEntry during build:index:source to eliminate the runtime repo dependency entirely.

3.3 Test Case Generator

Model	Gemini Pro (dev) / Claude Opus (prod) — deliberately different from the Code Generator to avoid shared blind spots in interpretation
Status	🛠 Planned — design complete, not yet implemented

The Test Case Generator runs in parallel with the Medplum Expert during the Plan phase. By the time the Code Generator runs, the test suite is already in state. It is written entirely from the ticket spec with no knowledge of the implementation that will follow. This is the independence guarantee: tests cannot be written to match code that does not yet exist.

Inputs — all from state

- issue — the raw GitHub issue body, carrying the full original spec, business context, and any edge cases or rules that may not have been fully captured in decomposition
- subtasks — structured acceptance criteria produced by the Issue Analyzer

Design decisions

- **Spec-derived, not implementation-derived:** tests are generated from acceptance criteria only. The Code Generator receives them as a constraint, not a scaffold.
- **Different model than Code Generator:** using a different model (e.g. Gemini vs Claude) reduces the risk that both agents share the same blind spots or make the same assumptions about edge cases.
- **Structured file output:** returns { filePath, content }[] consistent with the Code Generator output contract — one test file per subtask, written to the workspace.
- **The testing stack is fixed:** Vitest + React Testing Library. Conventions are injected via the prompt rather than fetched at runtime because the stack is known and stable.

Prompt design

- **Role:** You are a senior QA engineer writing Vitest test cases for a React component. You write tests from acceptance criteria only because you have no knowledge of the implementation. Tests should be red until correct implementation code exists.
- **Context block:** the subtask title, description, and acceptance criteria from state.
- **Output contract:** return a JSON array of { filePath, content }[]. One test file per subtask. Tests must cover all acceptance criteria, edge cases, and error states described in the spec.

3.4 Code Generator

Model	Claude Sonnet (prod) / Gemini Pro (dev) — highest-capability tier for the core generation step
Status	🛠 Planned — design complete, not yet implemented

The Code Generator is the core value step. It receives the full planning context assembled by the preceding agents and produces typed TypeScript file outputs targeting the existing codebase. Everything upstream exists to make this step as constrained and accurate as possible.

Inputs — all from state

- subtasks — ordered task list with titles, descriptions, acceptance criteria, and explicit dependencies, produced by the Issue Analyzer
- medplumContext — selected source entries, source snippets, FHIR schemas, and narrative briefing, produced by the Medplum Expert
- repoMap — target codebase file structure, produced by the Repo Map Generator
- testCases — pre-generated test suite written from spec, produced by the Test Case Generator

Design decisions

- **Sequential per-subtask generation:** subtasks carry explicit dependencies. The Code Generator processes them in dependency order to ensure earlier outputs are available as context before dependent subtasks are generated. This approach reduces prompt size per call and makes retry targeting precise.
- **Explicit file list:** each generation call includes the exact files expected as output. This constrains scope and makes off-scope writes detectable.
- **Structured file output:** the agent returns a JSON array of { filePath, content } objects rather than free-form text. This makes downstream file writes deterministic.
- **Tests are independent:** the test suite arrives pre-generated from spec before any implementation exists. The Code Generator does not write tests. Instead, it writes implementation code that the pre-existing tests will validate. This is the primary guarantee that QA cannot be gamed by the generator.

Prompt design

The Code Generator prompt is structured in three parts:

- **Role:** You are a senior React developer working on a Medplum-based clinical application. You write typed TypeScript components and hooks that integrate with Medplum's React SDK and FHIR data layer. You do not invent components or hooks. You use only what is provided in the context.
- **Context block:** This is injected from the state and includes the subtask being generated, the relevant source snippets and FHIR schemas from medplumContext, the repo map showing where files should be written, and the test cases the implementation must satisfy
- **Output contract:** Return a JSON array of { filePath, content } objects only. Provide one entry per file. File paths must match the repo map. Do not write files outside the expected list.

3.5 Code Quality Check

Model	None — deterministic script node, no LLM
Status	🛠 Planned — design complete, not yet implemented

The Code Quality Check is not an LLM agent; it is a deterministic script node. It runs ESLint, TypeScript compilation, and modularity checks against the generated code and produces structured, machine-readable output. No LLM is involved at this step, which is deliberate. Deterministic tools catch real errors with exact file paths and line numbers, making retry feedback precise and unambiguous.

Inputs — all from state

- generatedCode — the file outputs produced by the Code Generator

Design decisions

- **No LLM in the gate:** ESLint and tsc produce exact, structured errors. An LLM code review at this step would add non-determinism and cost where certainty matters most.
- **Full error output fed to retry:** on failure, the complete structured error output (file, line, message) is written to state and passed back to the Code Generator. The Code Generator decides what to fix while this node only reports.
- **Limited retry maximum:** bounds compute cost and prevents stuck loops. If the retry also fails, the pipeline exits and posts a structured comment to the issue with the exact failure and action needed.
- **Results written to state:** the quality check result is stored for the PR Generator to include in the Known Gaps section.

Retry routing

On failure: error output written to state, status set back to running, Code Generator re-invoked with errors as additional context. On retry failure: status set to failed, structured comment posted to issue, pipeline stops.

3.6 QA Agent

Model	Phase 1: None. Phase 2: Gemini Pro (dev) / Claude Opus (prod) — deliberately different from the Code Generator
Status	🛠 Planned — design complete, not yet implemented

The QA Agent runs in two phases. Phase 1 executes the pre-generated test suite — deterministic, no LLM, pass/fail against spec. Phase 2 takes an adversarial stance: an LLM reviews the generated code and

attempts to find what the spec-derived tests did not cover, generating and running additional test cases targeting edge cases, error boundaries, and unexpected inputs.

Inputs — all from state

- testCases — pre-generated Vitest test files from the Test Case Generator
- generatedCode — implementation files from the Code Generator

Design decisions

- Two-phase execution: Phase 1 validates the spec. Phase 2 tries to break what Phase 1 passed. Both phases must pass before the pipeline continues.
- Adversarial model choice: Phase 2 uses a different model than the Code Generator deliberately — to avoid shared blind spots. A model that generated the code is more likely to make the same assumptions when reviewing it.
- Structured test output fed to retry: on failure in either phase, the full structured output (failing test name, assertion, trace) is written to state and passed back to the Code Generator. One retry maximum across both phases.
- Results written to state: both phase results are stored for the PR Generator to include in the quality gate summary and Known Gaps sections.

Retry routing

On failure in either phase: structured output written to state, status set back to running, Code Generator re-invoked with failing traces as additional context. On retry failure: status set to failed, structured comment posted to issue with exact failures and action needed, pipeline stops.

Prompt design

- **Role:** You are an adversarial QA engineer reviewing generated code for a Medplum-based clinical application. Your job is not to validate what works; Phase 1 already did that. Your job is to find what the spec-derived tests did not cover: edge cases, error boundaries, unexpected inputs, and failure modes the original requirements did not anticipate. You are trying to break the implementation.
- **Context block:** Injected from state and includes the generated implementation files, the Phase 1 test results and coverage summary, the original subtask list and acceptance criteria, and the FHIR schemas for the resource types involved to inform clinically realistic edge cases.
- **Output contract:** Return a JSON array of { testName, testCode, rationale } objects only. Each entry is a single Vitest test case. Do not re-test anything already covered by Phase 1. Do not modify the implementation files. Rationale must explain why this edge case was not covered by the spec-derived tests.

3.7 PR Generator

Model	Claude Haiku — summarisation task, lower capability required
Status	🛠 Planned — design complete, not yet implemented

The PR Generator is the human handoff step. It assembles a structured pull request description from all run artifacts and posts it along with the generated code. The PR description is what makes AI-generated code reviewable rather than just mergeable.

Inputs — all from state

- issue — he original GitHub issue, used to link the PR back to the work order and provide reviewer context
- subtasks — the ordered task list produced by the Issue Analyzer, used to map what was built to what was asked
- generatedCode — file outputs from the Code Generator, committed to the branch
- qualityCheckResult — ESLint and tsc results, surfaced in the quality gate summary
- qaResult — test run results from both QA phases, surfaced in the quality gate summary
- logs — the full agent execution trail, included for traceability

PR description structure

- **What was built:** mapping from subtasks to files changed
- **How to test:** steps derived from acceptance criteria
- **Known Gaps:** mandatory section listing assumptions made, edge cases skipped, and anything the harness was uncertain about sourced from log entries, autocorrection warnings from the Medplum Expert, and any quality gate retry history
- **Quality gate results:** TypeScript compiled, ESLint passed, tests passed
- **Run log:** full agent execution log for traceability

Design decisions

- **Summarisation only — no generation:** the PR Generator does not produce code or make decisions. It reads what other agents produced and writes a human-readable account of it. Haiku is sufficient because the task is synthesis and formatting, not reasoning.
- **Known Gaps is mandatory:** the section is always present, even on a clean run. An empty Known Gaps section signals a clean run explicitly. A populated one signals where the harness had to make assumptions or recover from errors. This is what builds reviewer trust over time — it signals the harness is aware of its own limitations rather than presenting incomplete work as finished.
- **GitHub integration via Octokit:** the agent authenticates with the GitHub API using repository credentials from the environment, creates a branch, commits the generated files from state.generatedCode, and opens the PR against the target branch. Labels such as ai-generated and needs-review are applied at creation time.

- **Status set to done on success:** the PR Generator is the terminal node. Setting status to done signals that the pipeline has completed its full cycle and a human reviewer is now in the loop.

Prompt design

- **Role:** You are a senior engineer writing a pull request description for a code review. You are summarising work produced by an automated pipeline, not writing the code yourself. Be precise and honest. If the pipeline made assumptions or recovered from errors, say so.
- **Context block:** injected from state and includes the original issue title and body, the subtask list with acceptance criteria, the quality gate results summary, and any correction or truncation warnings recorded in the run logs.
- **Output contract:** Return a single markdown string formatted as a GitHub pull request body. Include all five sections: What was built, How to test, Known Gaps, Quality gate results, and Run log. Do not omit any section.

4. Testing

Testing is layered across four levels: unit tests for individual agents, graph wiring tests for LangGraph routing, groundedness suites for hallucination and structural integrity, and end-to-end pipeline runners against live LLMs. The test runner throughout is Vitest.

4.1 Unit Tests

All LLM calls and file I/O are mocked. No API key or Medplum repo is required to run the unit suite. This means tests run in CI without external dependencies.

Issue Analyzer — issueAnalyzer.test.ts

Seven mock issues were written and validated: three intentionally bad (no acceptance criteria, missing business rule, contradictory criteria), two with technical ambiguity that should pass, and two original drug interaction checker issues. All seven returned the correct decision. This included a tricky vitals ticket that looked well-formed but had an undefined business rule, as "abnormal" was never defined.

- Mocks the LLM via vi.mock to return controlled pass and reject responses
- Validates that a PASS result populates state.subtasks correctly
- Validates that a REJECT result sets status to needs_clarification and populates rejectionReason
- Validates that an LLM error sets status to failed and writes a log entry
- Validates immutability: incoming state.logs is never mutated.

Medplum Expert — medplumExpert.test.ts

- Mocks fs module to provide synthetic source and FHIR indexes without reading disk
- Validates the three-pass sequence: source filter → FHIR filter → summary generation
- Verifies that selectedSourceEntries and selectedFhirSchemas are populated in the output MedplumContextDocument
- Tests failure paths: missing index files and failed LLM calls both set status to failed
- Validates immutability: incoming state.logs is never mutated.

4.2 Graph Wiring Tests

These tests verify LangGraph routing — that the graph routes correctly between nodes and that state flows as expected. Agents themselves are mocked; only the graph structure is under test.

Graph wiring — graph.test.ts

- Mocks issueAnalyzer and medplumExpert as vi.fn()
- Verifies a passing issue (status = "running") routes through to medplumExpert
- Verifies a rejected issue (status = "needs_clarification") routes to END without invoking medplumExpert
- Verifies a failed issue routes to END

Routing logic — edges.test.ts

- Unit tests for routeAfterIssueAnalyzer with all three status variants
- Verifies needs_clarification → END, failed → END, running → "medplumExpert"

4.3 Groundedness and Structural Integrity Suites

These suites use live LLM calls to stress-test adversarial inputs. They are isolated from CI to prevent flaky pipelines from live model variance and run manually before shipping.

Issue Analyzer — issueAnalyzer.hallucination.test.ts

Tests structural integrity of the subtask graph the agent produces:

- DAG integrity: every declared dependency title matches an actual generated subtask title. The agent cannot reference a task that does not exist.
- Cycle detection: the dependency graph is verified to be acyclic using DFS. The orchestrator requires a valid DAG to execute tasks in order.

Medplum Expert — medplumExpert.hallucination.test.ts

Tests two layers of groundedness:

- Layer 1 — structured output: every entry in selectedSourceEntries must exist in the real source index; no type-only entries are selected; every FHIR schema name exists in the FHIR index
- Layer 2 — summary text: the free-text summary must not mention any Medplum export name that was not in the selected entries for that run. This catch ensures that a hallucinated ID, even if dropped from structured output, does not leak into the summary seen by the Code Generator.

Regression tests pin two specific known failure modes:

- react/ResourceForm must never appear in selected entries (this export does not exist in the index)
- Wrong-package-prefix entries (e.g. core/convertIsoToLocal) must be autocorrected to the right prefix rather than silently dropped
- List-returning tasks must select useSearchResources, not useSearch

4.4 Integration Runners

These scripts run the compiled agents against live LLMs and real index files. They are not unit tests and are not run in CI. They cost tokens and depend on live API access.

- runIssueAnalyzer.ts: runs all webhook payloads through the Issue Analyzer with a live LLM, printing subtask decompositions and rejection comments
- runMedplumExpert.ts: runs pre-configured fixture states through the Medplum Expert with real indexes, printing selected entries and the generated summary
- runHarnessE2E.ts: runs the full compiled LangGraph against all webhook payloads sequentially. It logs agent decisions, final status, and assembled Medplum context dimensions. This is the primary artefact that validates that the two implemented agents work together as a system.

4.5 Test Philosophy

- Unit tests mock everything external. The goal is to test agent logic in isolation, not the LLM provider. LLM behaviour is covered by separate groundedness suites.
- Groundedness suites are isolated from CI. Live model responses are non-deterministic; running them in CI creates flaky pipelines. They run manually before shipping.

- Tests are spec-derived, not implementation-derived. For the planned Code Generator, test files will be generated from ticket acceptance criteria before any implementation code is written. This is the primary guarantee that QA is independent.
- Graph wiring tests cover routing, not agents. The graph tests confirm routing logic; they don't re-test what agent unit tests already cover.

5. Medplum Context Design Rationale

5.1 Why Not RAG

RAG is appropriate when the knowledge base is large, grows continuously, or changes unpredictably. For Medplum's React SDK, the relevant frontend API surface is bounded: roughly 15 hooks and 30–40 components a typical developer actually reaches for. This fits comfortably in a structured index that can be filtered by an LLM in a single pass. RAG adds retrieval failures and infrastructure for a problem that doesn't yet require it. It's the natural evolution if the harness needed to work across an unknown or much larger codebase.

5.2 Why Not Custom MCP

A custom MCP server wrapping the indexes would allow the Code Generator to look things up on demand rather than having context injected upfront. This is a cleaner production architecture because the agent fetches exactly what it needs mid-generation. It was not built here because the Domain Expert runs once per ticket, the context is known before generation starts, and the tool call loop inside LangGraph adds real complexity. If the harness grew to support multiple target codebases, a custom MCP server over a RAG system would be the right next step.

6. Quality Gates

Quality gates are what stand between "the agent wrote code" and "a human sees a PR." The gate sequence is: Code Quality (ESLint + tsc) → Vitest test suite → PR Generator.

6.1 Automated Checks

- ESLint — code style and error patterns, including import validation
- tsc --noEmit — TypeScript type checking in strict mode. Catches hallucinated imports and type mismatches that would fail at runtime
- Vitest — runs the test suite generated before code was written, so tests are independent of implementation

6.2 Self-Correction

On quality gate failure, the full error output including the exact file, line, and error message is fed back to the Code Generator. It is given a constraint to fix only the failing issues without touching unrelated code. This process is limited to a set number of retries. If the final retry also fails, the harness posts a structured comment to the issue explaining what failed and what action is needed before re-queuing.

6.3 Test Independence

Tests are generated from the ticket spec before any code is written. This is the primary guarantee that QA is independent because tests cannot be written to match code that does not yet exist. Tests are derived from the spec rather than from the implementation.

6.4 Human Handoff

The PR description is the human handoff artifact. The Known Gaps section is mandatory; it lists assumptions made, edge cases skipped, and anything the harness was uncertain about. This is what makes AI-generated code reviewable rather than just mergeable, and it is what builds reviewer trust over time.

7. Failure Modes and Observability

Failure Mode	Detection	Response
Ticket too vague	Issue Analyzer rejects at gate 1	Clarification comment posted to issue; pipeline stops
Technical gap mistaken for product gap	Classification step in Issue Analyzer prompt separates the two before deciding	Technical gaps pass through — Medplum Expert resolves them downstream
Context window overflow	Medplum Expert tracks token count of assembled context	Truncate to most relevant entries; flag in PR Known Gaps section
Hallucinated API usage	tsc catches unknown identifiers; ESLint catches undefined imports	Error fed to Code Generator on retry with specific line numbers
Agent goes off-scope	Subtask list constrains output; Code Generator prompted with explicit file list	Files outside expected list trigger scope warning in log
Retry also fails	retryCount === 1 and gate still failing	Give-up comment posted to issue with exact failure and action needed
LLM API unavailable	HTTP error on API call	Retry once with exponential backoff, then fail with error logged

7.1 Observability

A LogEntry is written to state.logs at each agent boundary. Every entry records the agent name, timestamp, decision made, and success/failure status. This is enough to replay what happened on any given run and identify where a pipeline failed.

```
interface LogEntry {
  agent: string;
  timestamp: string;
  decision: string;
  status: "success" | "failure";
}
```

Production evolution: LangSmith integration for full trace UI, OpenTelemetry spans for existing observability stacks, cost-per-ticket tracking, and success/retry/failure rates per agent to direct prompt improvement investment.

8. Deliberate Tradeoffs

Decision	Rationale
Static indexes vs RAG	RAG adds infrastructure and retrieval failures for a bounded, well-defined domain. Static injection is simpler, faster, and fully inspectable. RAG is the natural evolution if the harness needs to work across unknown codebases.
Static indexes vs custom MCP server	The Domain Expert runs once per ticket and is not an interactive agent making dynamic queries. Static files work well for a bounded API surface. A custom MCP server would be the right V2 if the harness expanded to multiple target codebases.
Three-pass Medplum Expert vs one-shot	The full source index and FHIR schema list together are large. Pass 1 and 2 use them as lightweight filters. Pass 3 injects only the relevant pieces. The Code Generator receives focused context rather than noise.
Limited retry vs unlimited	Bounds compute cost and prevents stuck loops. The give-up comment makes the issue re-queueable with clear action needed.
No LLM in quality gate	Deterministic tools produce machine-readable errors suitable for retry feedback. LLM code review adds non-determinism and cost at a step where certainty matters most.

9. Future Work

- Complete Code Generator, Code Quality Gate, and PR Generator to close the MVP loop
- Custom MCP server over the static indexes (if the compression pipeline is insufficient): allows on-demand lookup rather than upfront injection; a natural evolution as the harness expands to more codebases
- RAG layer under the MCP server (if the compression pipeline is insufficient): for unknown or large codebases where static indexing is not sufficient
- GitHub Actions trigger: currently the pipeline runs locally with issue.json as input; wiring to a real GitHub event completes the agentic SDLC loop
- LangSmith integration for full trace observability and run replay
- Sequential per-subtask generation with repo map regeneration between subtasks: reduces type mismatches on longer task lists
- Cross-provider test generation (different model for tests and code) to reduce shared blind spots
- Automated knowledge base sync: weekly workflow rebuilds the Medplum index and opens a PR for human review before merging